Lab experiment-13

Minimax Algorithm for Gaming

Aim

To write a Python program to implement the Minimax algorithm for a simple game**.**

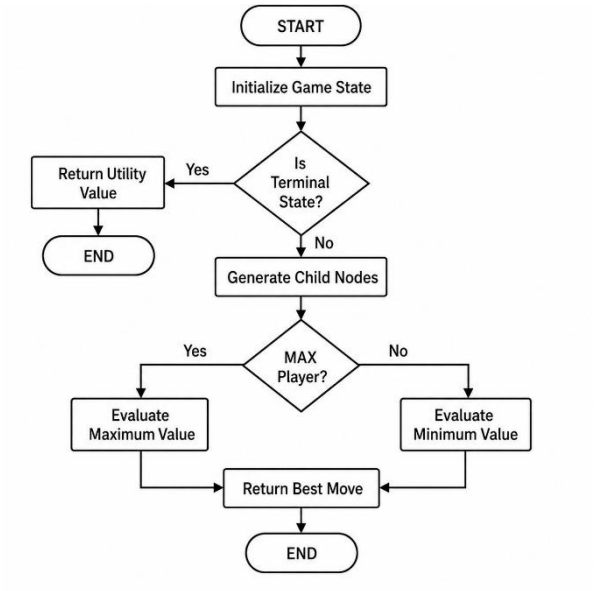
Objective

To understand the Minimax decision-making algorithm.

Algorithm

1. Start.
2. Initialize the game state.
3. Generate all possible moves.
4. Evaluate each possible move using Minimax.
5. Choose the move with the highest score for the AI.
6. Repeat until the game ends.
7. Display the winner.
8. Stop.

Flowchart

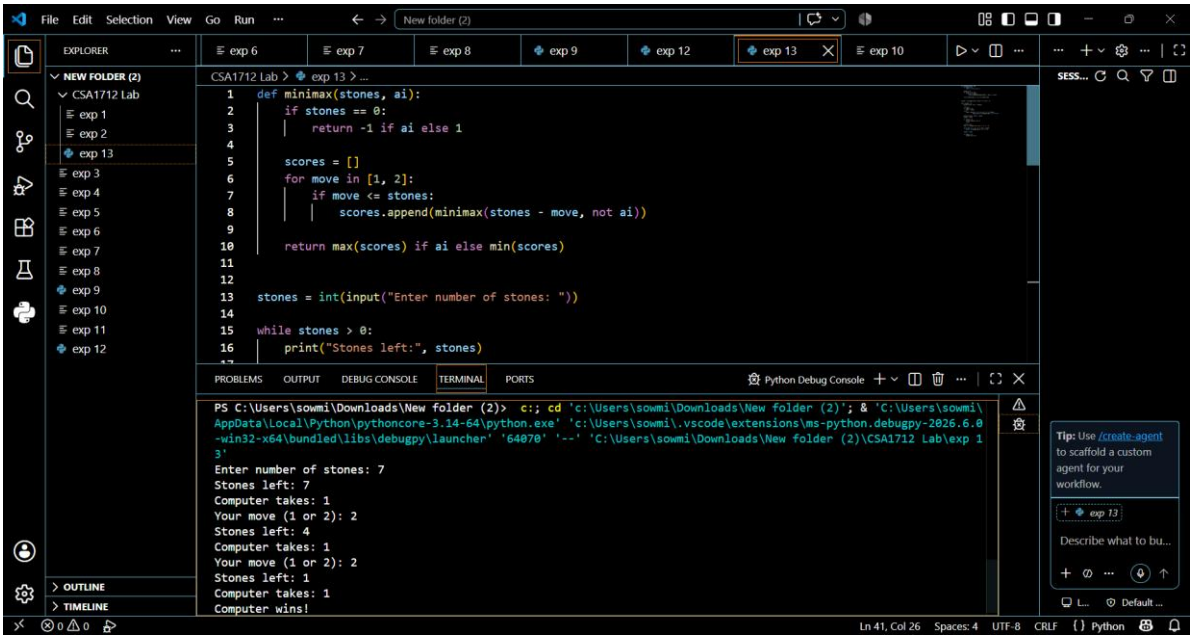


Code

```
def minimax(stones, ai):
    if stones == 0:
        return -1 if ai else 1
    scores = []
    for move in [1, 2]:
        if move <= stones:
            scores.append(minimax(stones - move, not ai))
    return max(scores) if ai else min(scores)
stones = int(input("Enter number of stones: "))
while stones > 0:
    print("Stones left:", stones)
    # AI move
    if stones <= 2:
        move = stones
```

```
else:
    move = max([m for m in [1, 2] if m <= stones],
               key=lambda m: minimax(stones - m, False))
print("Computer takes:", move)
stones -= move
if stones == 0:
    print("Computer wins!")
    break
# Player move
move = int(input("Your move (1 or 2): "))
while move not in [1, 2] or move > stones:
    move = int(input("Enter 1 or 2: "))
stones -= move
if stones == 0:
    print("You win!")
```

Output



Result
The game was implemented successfully using the Minimax algorithm.

Conclusion
The Minimax algorithm helps the computer select the best possible move by evaluating future game states.